Costruzione di una Occupancy Grid 2D da dati LiDAR e ZED

Alessio Poggesi

2 maggio 2026

Indice

1 Introduzione **2**

2 Preprocessing dei dati **3**

3 Costruzione della occupancy grid **4**
3.1 Parametri di mapping 5

4 Struttura del codice e scelte implementative **5**

5 Limitazioni dei dati acquisiti **6**

6 Caso studio: mappa indoor **6**
6.1 Mappa probabilistica 7
6.2 Mappa classificata 8
6.3 Effetto della risoluzione 9
6.4 Analisi step-by-step con viewer 11
6.5 Esempi aggiuntivi in ambiente esterno 12

7 Conclusioni **14**

1 Introduzione

L'obiettivo di questo progetto è costruire una *occupancy grid* 2D offline a partire da dati reali acquisiti da un rover dotato di camera ZED e sensore LiDAR. Una *occupancy grid* è una rappresentazione discreta dell'ambiente, in cui lo spazio viene suddiviso in celle e a ciascuna cella viene associata una stima di occupazione.

Il contesto sperimentale è quello di un rover in scala utilizzato in ambiente controllato, equipaggiato con NVIDIA Jetson AGX Orin, stereo camera ZED 2i e LiDAR planare 2D. L'integrazione tra i dispositivi avviene tramite ROS2, che gestisce i diversi nodi e topic del sistema. Durante le sessioni di acquisizione, il rover viene guidato manualmente e i dati dei sensori vengono registrati in bag ROS2. Le registrazioni sono accompagnate da metadati YAML e da un database contenente i messaggi effettivi acquisiti.

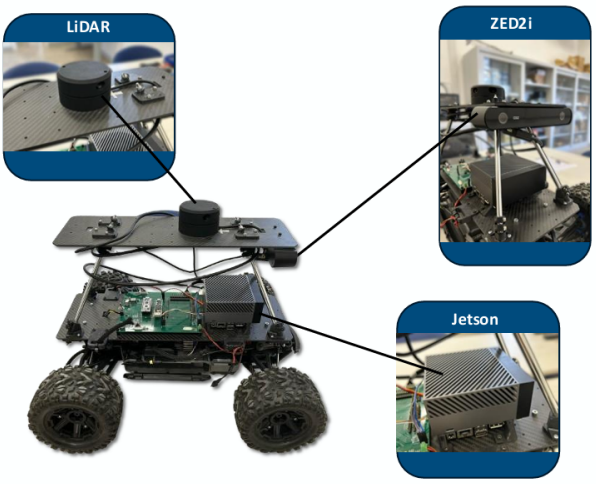


Figura 1: Raffigurazione del rover impiegato e dei principali componenti.

Ai fini della costruzione della mappa, non vengono utilizzati tutti i dati disponibili, ma solo i topic strettamente necessari:

- `/scan`, che contiene le scansioni LiDAR;

bag_time_ns (int64)	angle_min (rad)	angle_max (rad)	angle_increment (rad)	range_min (m)	range_max (m)	ranges (array di float32)
1733232016782891723	-3.14159	3.14159	0.006982	0.010	64.000	[0.0, 0.10, 0.55, 2.30, ..., 5.66, 0.0, 0.0]
...	...	...	...	...	...	...

Figura 2: Esempio di dati raccolti dal topic `/scan`.

- `/zed/zed_node/odom`, che contiene i dati di odometria della ZED.

bag_time_ns (int64)	position.x (float)	position.y (float)	position.z (float)	orientation.x (float)	orientation.y (float)	orientation.z (float)	orientation.w (float)
1733232017658247386	47.423633	38.262071	0.971456	0.033273	0.088079	0.220672	0.971456
1733232018034295879	47.925014	28.459045	0.977451	0.036671	0.079720	0.193114	0.977451
1733232018403404458	48.553671	28.644752	0.985039	0.035037	0.081316	0.147665	0.985039
...	...	...	...	...	...	...	...

Figura 3: Esempio di dati raccolti dal topic `/zed/zed_node/odom`.

A partire da questi dati, il workflow del progetto si articola in tre livelli principali:

- **preprocessing**, in cui vengono letti i bag, vengono allineati temporalmente i dati e vengono ricostruiti i punti LiDAR locali e globali;
- **mapping**, in cui tali punti vengono utilizzati per aggiornare la occupancy grid, marcando lo spazio libero lungo i raggi e lo spazio occupato nei punti finali;
- **debug e explainability**, in cui vengono prodotti plot, riepiloghi numerici e viewer HTML per analizzare la qualità della mappa, il comportamento dei raggi a zero e l'effetto dei parametri.

Anche il codice è stato organizzato in modo modulare, separando preprocessing, costruzione della mappa, test degli esperimenti e strumenti di analisi visuale. Questa separazione rende il workflow più chiaro, verificabile ed estendibile.

2 Preprocessing dei dati

Il preprocessing ha lo scopo di trasformare i dati grezzi del bag ROS2 in una rappresentazione coerente e pronta per la fase di mapping. Il file principale per questa fase è `preprocess_bag.py`.

La pipeline di preprocessing può essere riassunta nei seguenti punti:

- **Lettura dei dati.** Dal bag ROS2 vengono estratti i messaggi di odometria e le scansioni LiDAR, che costituiscono gli input minimi del mapping.
- **Gestione dei timestamp.** Per ogni messaggio vengono mantenuti sia il timestamp di registrazione nella bag, indicato come `bag_time_ns`, sia il timestamp interno del messaggio ROS, indicato come `header_time_ns`. Il codice introduce inoltre un campo operativo `time_ns`, che corrisponde al timestamp scelto per la sincronizzazione. La scelta è parametrica tramite `timestamp_source`, che può assumere i valori `bag_time` oppure `header_time`. Nella versione attuale il default utilizzato è `header_time`, perché risulta più vicino all'istante dichiarato di acquisizione dei sensori.
- **Trattamento della posa.** L'orientamento, fornito in forma di quaternione dall'odometria, viene convertito in yaw, così da lavorare nel piano usando soltanto posizione x, posizione y e orientamento planare.
- **Allineamento temporale e stima della posa.** Poiché odometria e scansioni non sono perfettamente sincrone, per ogni scan viene stimata la posa ZED al tempo `scan.time_ns`. Il codice cerca i due campioni odometrici che racchiudono temporalmente lo scan e interpola linearmente x e y. Lo yaw viene invece interpolato usando una differenza angolare normalizzata, così da evitare errori nel passaggio tra $-\pi$ e $+\pi$.
- **Trasformazione ZED–LiDAR.** Poiché la posa disponibile è riferita alla ZED ma le misure provengono dal LiDAR, viene applicata una trasformazione rigida approssimata tra i due sensori. Nella versione attuale del codice, prima viene applicata una correzione di yaw alla ZED pari a:

  $$\Delta\theta_{zed} = \pi$$

  e poi viene composto un offset planare ZED–LiDAR:

  $$T_{\text{zed}\to\text{lidar}} \approx (0.15,\ 0.0,\ 0.0)$$

  Questi valori devono essere interpretati come una calibrazione empirica della pipeline, non come una misura metrologica definitiva della posizione fisica dei sensori. In particolare, la correzione di yaw pari a π è stata introdotta perché la mappa risultava ruotata/invertita, e la sua applicazione ha migliorato sensibilmente la coerenza geometrica della ricostruzione.

- **Filtraggio e conversione delle misure.** Prima della conversione vengono scartati i valori LiDAR non validi: valori nulli, non finiti, minori o uguali alla soglia minima del sensore, superiori al `range_max` dichiarato dal messaggio oppure superiori al limite operativo scelto. Nella versione attuale è stato introdotto un massimo effettivo:

  $$\texttt{effective_range_max} = 12.0 \text{ m}$$

  anche se il messaggio LaserScan dichiara un `range_max` più alto. Le misure valide vengono convertite da coordinate polari (r, θ) a punti cartesiani locali:

  $$x = r\cos(\theta), \qquad y = r\sin(\theta)$$

  e successivamente trasformate nel frame globale `odom`.

- **Output intermedio.** Il risultato finale è una struttura dati che, per ogni scansione allineata, contiene timestamp, posa ZED, posa LiDAR, punti locali e punti globali, pronta per la fase successiva di mapping.

L'output del preprocess non rappresenta ancora la occupancy grid. Contiene però tutte le informazioni necessarie per costruirla: le pose del LiDAR e gli endpoint globali dei raggi validi.

3 Costruzione della occupancy grid

A partire dall'output del preprocess, la fase di mapping costruisce una occupancy grid 2D nel frame globale `odom`, discretizzando lo spazio e aggiornando le celle in base alle osservazioni del LiDAR. Il file principale per questa fase è `build_occupancy_grid.py`.

La costruzione della mappa può essere riassunta nei seguenti punti:

- **Definizione dei bound spaziali.** La mappa viene costruita a partire da tutte le pose del LiDAR e da tutti gli endpoint globali dei raggi validi. Da questi dati vengono ricavati i valori minimi e massimi su x e y, a cui viene aggiunto un margine per evitare che i punti osservati cadano sul bordo.

- **Discretizzazione della mappa.** Una volta fissati i bound, lo spazio continuo viene discretizzato in una griglia 2D con risoluzione fissata. In questo modo, ogni punto del piano viene associato a una specifica cella della mappa.

- **Rappresentazione interna.** La griglia viene inizializzata in log-odds, con valore iniziale nullo, che rappresenta una condizione di incertezza sulle celle non ancora osservate. La probabilità iniziale implicita è quindi pari a 0.5.

- **Origine e fine del raggio.** Per ogni scansione viene considerata come origine la posa del LiDAR, mentre come punto finale viene usato ciascun endpoint globale ottenuto dal preprocess.

- **Ray tracing.** Tra la cella di origine e la cella finale del raggio viene applicato un algoritmo di Bresenham, così da individuare in modo discreto tutte le celle attraversate dal raggio.

- **Aggiornamento delle celle.** Le celle attraversate dal raggio, escluse la cella finale, vengono aggiornate come *free*; la cella finale viene aggiornata come *occupied*. Gli aggiornamenti sono additivi in log-odds e vengono saturati entro un intervallo minimo e massimo per evitare valori estremi.

- **Classificazione finale.** Al termine degli aggiornamenti, la griglia viene convertita da log-odds a probabilità e classificata in tre stati: libero, occupato o incerto, in base a due soglie fissate.

Il risultato finale è costituito da una mappa probabilistica, una mappa classificata e da una serie di file di output utili per il debug e il confronto tra esperimenti.

3.1 Parametri di mapping

Il file `build_occupancy_grid.py` definisce una configurazione di base tramite `OccupancyGridConfig`. Lo script di test `run_test_mapping.py` può però sovrascrivere alcuni valori per l'esperimento corrente. Nella configurazione di test allegata sono usati, in particolare:

```
CONFIG = OccupancyGridConfig(
    resolution=0.08,
    margin=1.0,
    p_occ=0.75,
    p_free=0.40,
    log_odds_min=-4.0,
    log_odds_max=4.0,
    free_threshold=0.35,
    occ_threshold=0.75,
    scan_stride=1,
    exclude_origin_cell_from_free=False,
)
```

Il significato dei parametri principali è il seguente:

- `resolution`: lato della cella in metri;
- `margin`: margine aggiunto ai bound della mappa;
- `p_occ`: probabilità modello associata a un endpoint osservato;
- `p_free`: probabilità modello associata a una cella attraversata dal raggio;
- `log_odds_min` e `log_odds_max`: limiti di saturazione;
- `free_threshold` e `occ_threshold`: soglie usate solo nella classificazione finale;
- `scan_stride`: permette di usare una scansione ogni N;
- `exclude_origin_cell_from_free`: stabilisce se escludere la cella del LiDAR dall'aggiornamento come libera.

4 Struttura del codice e scelte implementative

Il progetto è stato organizzato in moduli distinti, così da separare le diverse fasi del workflow e rendere il codice più leggibile, riutilizzabile ed estendibile.

La struttura del codice allegata può essere riassunta nei seguenti punti:

- **`preprocess_bag.py`.** Gestisce la lettura del bag ROS2, l'estrazione dei topic necessari, la gestione dei timestamp, l'interpolazione temporale, la stima della posa e la costruzione dei punti 2D nel frame globale.
- **`build_occupancy_grid.py`.** Si occupa della costruzione della mappa a partire dall'output del preprocess, includendo discretizzazione, ray tracing, aggiornamento in log-odds, classificazione finale e salvataggio degli output.
- **`run_test_mapping.py`.** Fornisce uno script semplice per lanciare gli esperimenti, modificare i parametri principali e salvare ogni run in una cartella dedicata.
- **`zero_ray_explainability_viewer_v2.py`.** Estende la pipeline con uno strumento di analisi visuale dei raggi LiDAR, in particolare dei raggi con valore zero e dei raggi oltre il massimo operativo. Lo script costruisce mappe step-by-step usando solo i raggi validi, sovrappone i raggi a zero come informazione grafica e genera una viewer HTML sincronizzata.

La scelta di separare i moduli è utile perché permette di verificare ogni fase in modo indipendente. In particolare, il mapping non rilegge direttamente il bag ROS2, ma lavora sull'output già prodotto dal preprocess. In questo modo la stessa struttura dati può essere riutilizzata sia per costruire la mappa finale, sia per generare visualizzazioni intermedie e strumenti di debug.

5 Limitazioni dei dati acquisiti

Durante l'analisi dei dati reali sono emerse alcune limitazioni che influenzano la qualità finale della occupancy grid. Queste limitazioni non indicano necessariamente un errore nell'algoritmo di mapping, ma dipendono soprattutto dalla qualità delle misure e dalla coerenza tra i frame dei sensori.

Misure LiDAR nulle Nelle scansioni LiDAR sono presenti numerosi valori pari a zero. Questi valori non vengono interpretati come ostacoli a distanza nulla, ma come misure non valide. Per questo motivo, nel preprocess vengono scartati insieme ai valori non finiti e alle misure inferiori o uguali alla distanza minima del sensore.

Informazione di intensità È stata valutata l'intensità del segnale LiDAR come possibile criterio aggiuntivo per distinguere misure valide e non valide. Tuttavia, nei dati analizzati essa risulta poco informativa, assumendo spesso valori molto semplici o quasi costanti.

Per questo motivo l'intensità non è stata integrata nel modello probabilistico: la validità dei raggi viene determinata principalmente dal valore contenuto in `ranges`.

Stima della posa e calibrazione tra sensori La posa utilizzata nel mapping deriva dall'odometria della ZED, mentre le misure geometriche provengono dal LiDAR. Poiché non è disponibile una calibrazione completa e verificata tra i due sensori, la trasformazione ZED–LiDAR è stata stimata in modo approssimato e trattata come parametro sperimentale della pipeline.

Un'ulteriore criticità osservata nei dati riguarda la stabilità della posa: in alcune registrazioni la traiettoria stimata presenta piccole traslazioni o discontinuità improvvise. Quando questo accade, anche i punti LiDAR vengono proiettati nel frame globale in posizioni non perfettamente coerenti. L'effetto finale può essere la comparsa di muri duplicati, spostamenti locali o deformazioni della mappa.

6 Caso studio: mappa indoor

Come primo caso di analisi qualitativa è stata scelta una registrazione acquisita in ambiente interno. Questa mappa è particolarmente utile come caso studio perché presenta una struttura abbastanza leggibile: si distinguono regioni libere attraversate dai raggi LiDAR, bordi occupati associati a pareti o ostacoli, e zone rimaste non osservate.

La mappa è stata costruita usando una risoluzione di $0.08\,m$, quindi ogni cella rappresenta un'area di circa $8\,cm \times 8\,cm$. Sono stati utilizzati tutti gli scan disponibili dopo il preprocessing: in totale 154 scan su 154.

La griglia finale ha dimensione 171×262 celle. Durante la costruzione della mappa sono stati effettuati 2944548 aggiornamenti di celle libere e 108134 aggiornamenti di celle occupate. Questo evidenzia la differenza tra le due tipologie di aggiornamento: per ogni raggio viene aggiornata come occupata solo la cella finale, mentre molte celle attraversate lungo il percorso del raggio vengono aggiornate come libere.

Parametro	**Valore**
Risoluzione	$0.08\,m$
Margine della mappa	$1.0\,m$
Scan disponibili	154
Scan utilizzati	154
Raggi LiDAR utilizzati	108134
Aggiornamenti free	2944548
Aggiornamenti occupied	108134
Dimensione griglia	171×262 celle

Tabella 1: Parametri principali del caso studio indoor.

Questo caso di studio viene utilizzato come riferimento per tre livelli di analisi. In primo luogo, viene osservata la mappa probabilistica, che conserva l'informazione quantitativa sulla probabilità di occupazione delle celle. In secondo luogo, viene analizzata la mappa classificata, che traduce tali probabilità nelle tre classi *unknown*, *free* e *occupied*. Infine, la stessa registrazione viene esaminata tramite viewer step-by-step, così da valutare il contributo delle singole osservazioni, l'evoluzione cumulativa della mappa e la distribuzione dei raggi non validi o pari a zero.

6.1 Mappa probabilistica

La prima rappresentazione prodotta dalla pipeline è la mappa probabilistica, in cui ogni cella contiene una stima continua della probabilità di occupazione. Questa mappa deriva dalla conversione finale dei valori in log-odds e conserva più informazione rispetto alla successiva mappa classificata: una cella può infatti risultare più o meno libera, più o meno occupata, oppure rimanere vicina allo stato iniziale di incertezza.

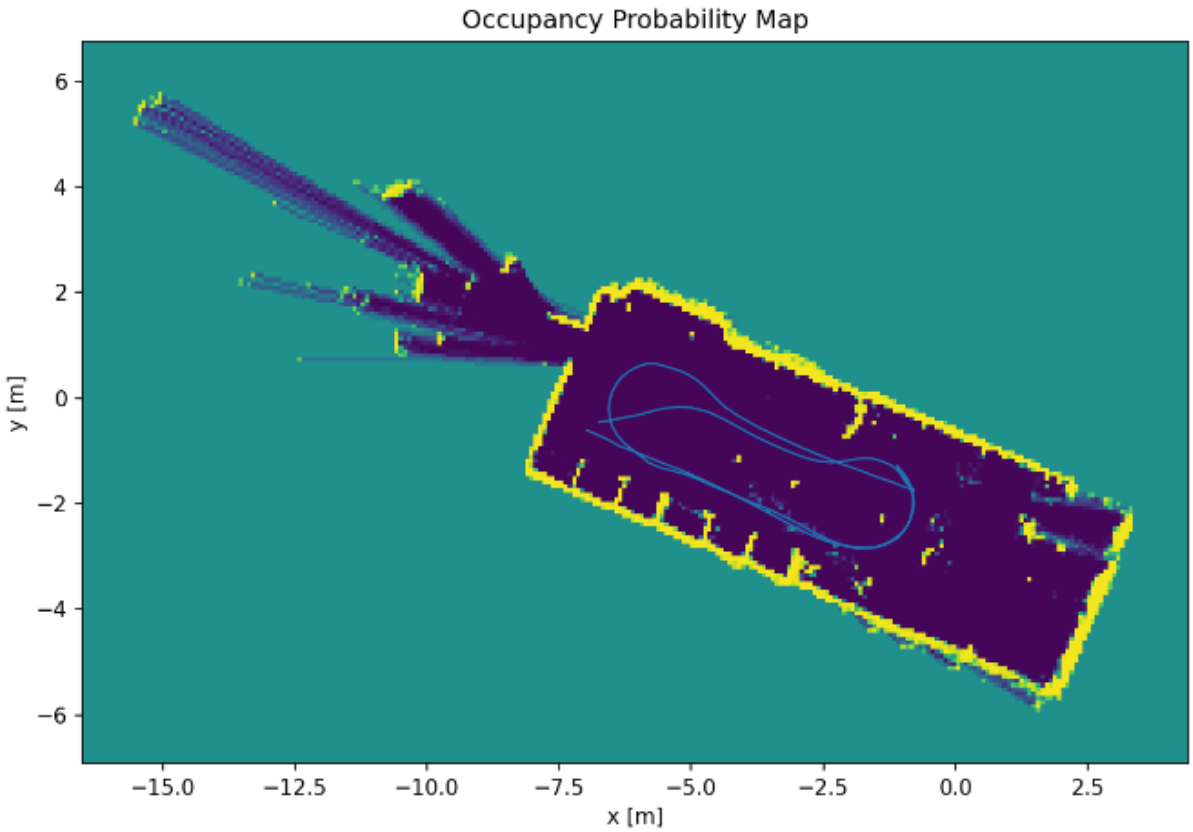


Figura 4: Mappa probabilistica ottenuta dalla registrazione indoor.

Nella Fig. 4 si osserva una regione principale a bassa probabilità di occupazione, corrispondente allo spazio attraversato dai raggi LiDAR durante il movimento del rover. Le celle a probabilità più alta si concentrano invece lungo i bordi dell'area osservata e rappresentano gli endpoint dei raggi, cioè le superfici interpretate come occupate.

La traiettoria sovrapposta aiuta a collegare la forma della mappa al percorso del sensore: le zone libere si sviluppano attorno al tragitto seguito, mentre le celle occupate delimitano pareti, ostacoli o altri elementi dell'ambiente interno. Le aree non attraversate da un numero sufficiente di raggi rimangono invece vicine allo stato di incertezza iniziale.

Nel complesso, la mappa mostra una struttura coerente con un ambiente indoor, pur presentando alcune irregolarità locali. Queste possono dipendere da rumore nelle misure, errori residui nella stima della posa o approssimazioni nella calibrazione tra ZED e LiDAR.

6.2 Mappa classificata

A partire dalla mappa probabilistica viene costruita anche una mappa classificata, in cui ogni cella viene assegnata a una delle tre classi finali: *unknown*, *free* oppure *occupied*. Nel caso considerato, una cella viene classificata come libera se la probabilità di occupazione è inferiore a 0.35, come occupata se è superiore a 0.75, e come incerta negli altri casi.

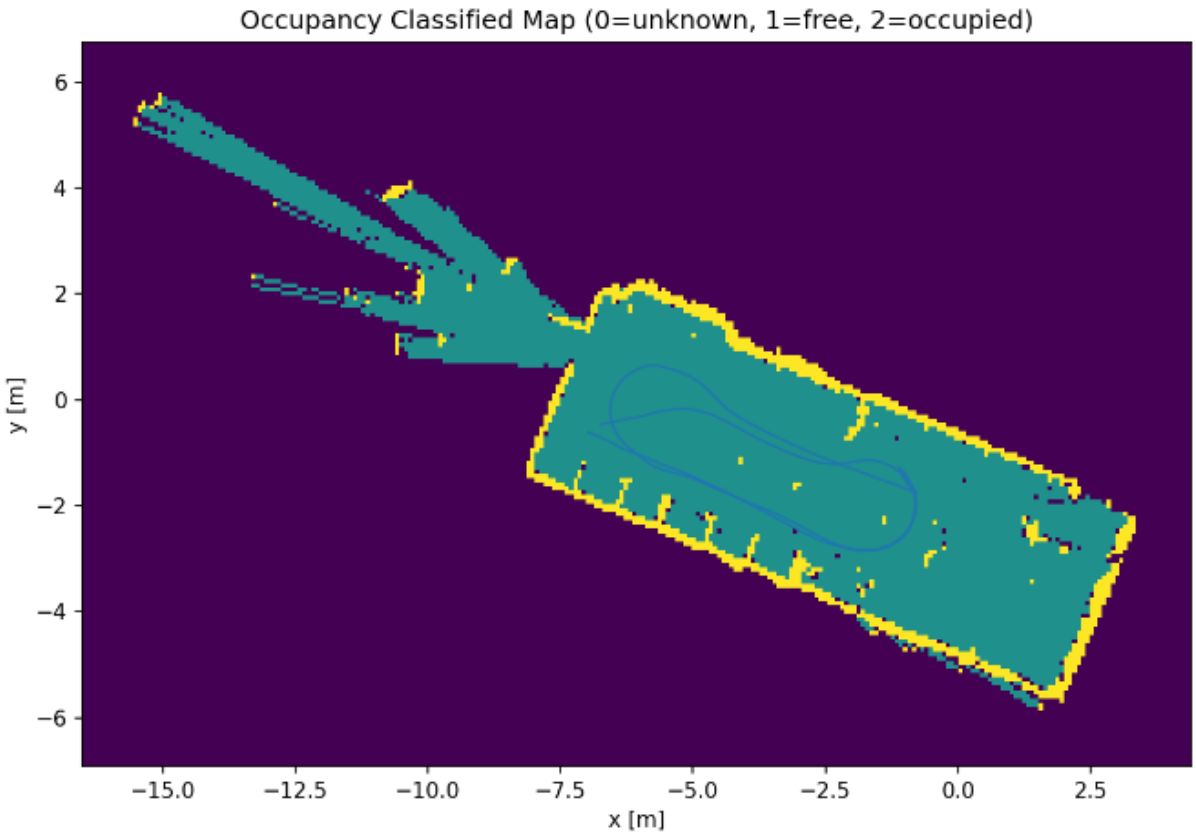


Figura 5: Mappa classificata ottenuta dalla registrazione indoor.

Nella Fig. 5 la struttura dell'ambiente risulta più immediata da interpretare rispetto alla mappa probabilistica. Le celle libere formano una regione continua lungo il percorso del rover, mentre le celle occupate si concentrano soprattutto lungo i bordi della zona osservata. Le celle rimaste non osservate o non sufficientemente informative vengono invece mantenute nello stato *unknown*.

Questa rappresentazione evidenzia bene il comportamento dell'algoritmo di mapping. Ogni raggio LiDAR contribuisce a liberare le celle attraversate e ad aumentare la probabilità di occupazione della cella finale. Dopo molte osservazioni, le regioni attraversate più volte diventano stabilmente libere, mentre gli endpoint ricorrenti formano contorni occupati più marcati.

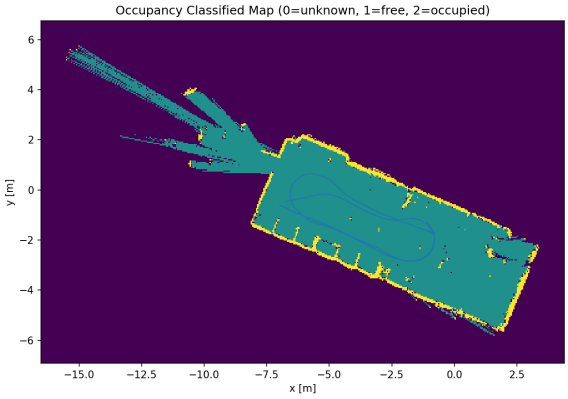
Rispetto alla mappa probabilistica, la mappa classificata è più leggibile e adatta a una valutazione qualitativa immediata. Tuttavia, essa perde parte dell'informazione quantitativa: celle con probabilità di occupazione diverse possono essere assegnate alla stessa classe finale. Per questo motivo le due rappresentazioni sono complementari: la mappa probabilistica è più informativa dal punto di vista numerico, mentre la mappa classificata è più chiara per osservare la struttura globale dell'ambiente.

Nel caso specifico, la mappa mostra una separazione abbastanza netta tra spazio osservato e spazio non osservato. Allo stesso tempo sono presenti alcune irregolarità, come bordi leggermente spessi o tratti duplicati. Questi effetti sono coerenti con una ricostruzione basata su dati reali e possono dipendere da errori residui nella posa, dalla calibrazione approssimata tra i sensori o dal movimento del rover durante l'acquisizione.

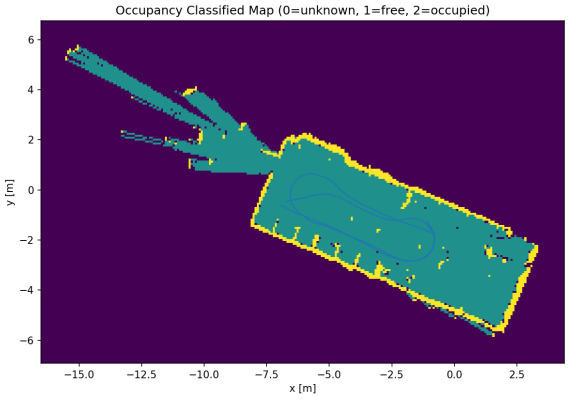
6.3 Effetto della risoluzione

Un parametro importante nella costruzione della occupancy grid è la risoluzione spaziale, cioè la dimensione fisica rappresentata da ogni cella. Questo valore influenza sia il livello di dettaglio della mappa sia il numero complessivo di celle da aggiornare.

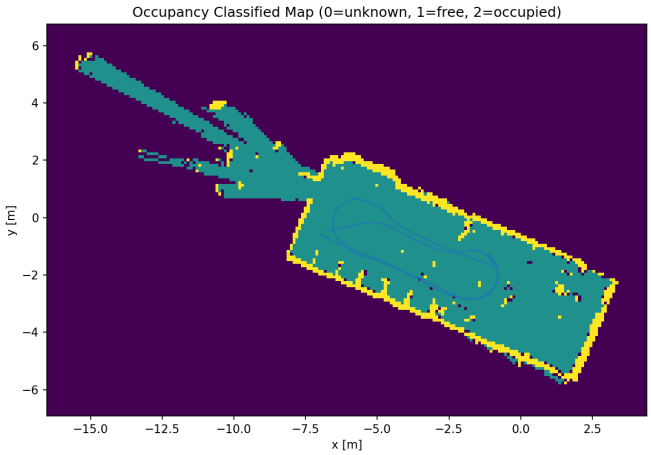
Per valutare l'effetto della discretizzazione, la stessa registrazione indoor è stata elaborata usando tre risoluzioni diverse: $0.05\,m$, $0.08\,m$ e $0.10\,m$, mantenendo invariati gli altri parametri principali del mapping.



(a) Risoluzione $0.05\,m$



(b) Risoluzione $0.08\,m$



(c) Risoluzione $0.10\,m$

Figura 6: Confronto tra mappe classificate ottenute dalla stessa registrazione con differenti risoluzioni spaziali.

Dal confronto in Fig. 6 si osserva il compromesso tipico delle occupancy grid. Con una risoluzione più fine, come $0.05\,m$, la mappa conserva più dettagli locali e rappresenta i contorni con maggiore precisione, ma rende anche più visibili rumore e piccole incoerenze dovute alla stima della posa o alle misure LiDAR.

La risoluzione $0.08\,m$, scelta come configurazione di riferimento, rappresenta un compromesso tra dettaglio e leggibilità: la struttura dell'ambiente rimane chiara senza aumentare eccessivamente la frammentazione della mappa.

Con una risoluzione più grossolana, come $0.10\,m$, la mappa risulta più compatta e meno dettagliata. Alcune irregolarità locali vengono attenuate, ma i contorni risultano meno precisi e le pareti possono apparire più spesse. In generale, quindi, la scelta della risoluzione dipende dal compromesso desiderato tra precisione geometrica, leggibilità e costo computazionale.

6.4 Analisi step-by-step con viewer

Per analizzare in modo più dettagliato la formazione della occupancy grid è stata utilizzata una viewer step-by-step. Questo strumento permette di osservare, per ogni scansione LiDAR, tre rappresentazioni affiancate: la mappa cumulativa aggiornata fino allo scan corrente, la mappa generata dalla sola osservazione corrente e la distribuzione dei raggi dello scan.

Questa visualizzazione è utile perché permette di separare il contributo locale di una singola misura dall'effetto globale prodotto dall'accumulo delle osservazioni. In una singola scansione, infatti, il LiDAR osserva solo una porzione limitata dell'ambiente. La mappa cumulativa, invece, integra progressivamente le informazioni acquisite lungo la traiettoria del rover.

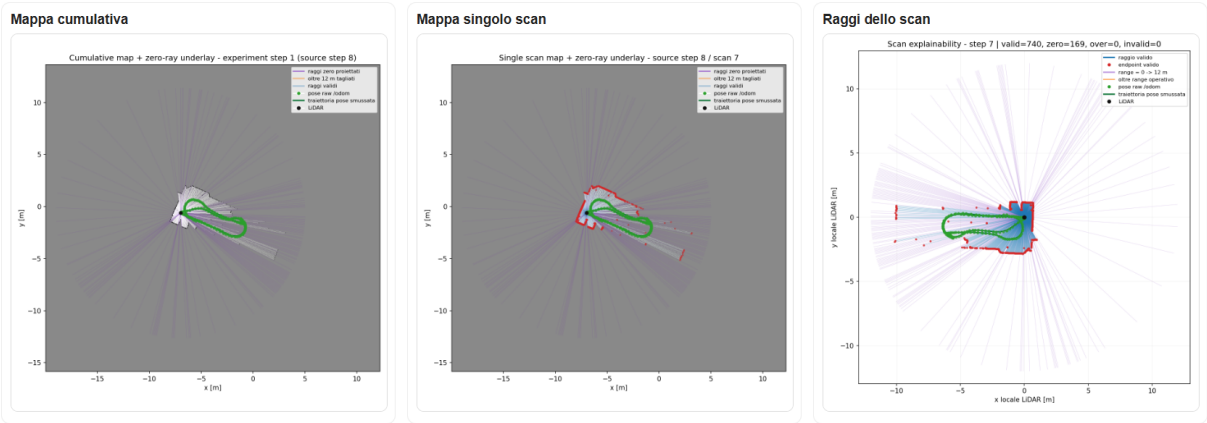


Figura 7: Viewer step-by-step in una fase iniziale della sequenza. A sinistra è mostrata la mappa cumulativa, al centro la mappa prodotta dalla singola osservazione, a destra la distribuzione dei raggi dello scan.

Nella Fig. 7 è mostrata una fase iniziale della costruzione della mappa. In questo punto la mappa cumulativa contiene ancora poche osservazioni e risulta quindi molto simile al contributo della singola scansione. La vista centrale mostra in modo evidente il comportamento del modello di aggiornamento: le celle attraversate dai raggi vengono aggiornate come libere, mentre gli endpoint validi dei raggi vengono interpretati come evidenza di occupazione.

La vista dei raggi permette inoltre di distinguere le diverse categorie di misure. I raggi validi sono quelli effettivamente utilizzati per aggiornare la occupancy grid. I raggi pari a zero, invece, vengono visualizzati graficamente ma non contribuiscono alla mappa, perché sono considerati misure non valide. Nell'esempio iniziale sono presenti 740 raggi validi e 169 raggi a zero, mentre non compaiono raggi oltre il range operativo.

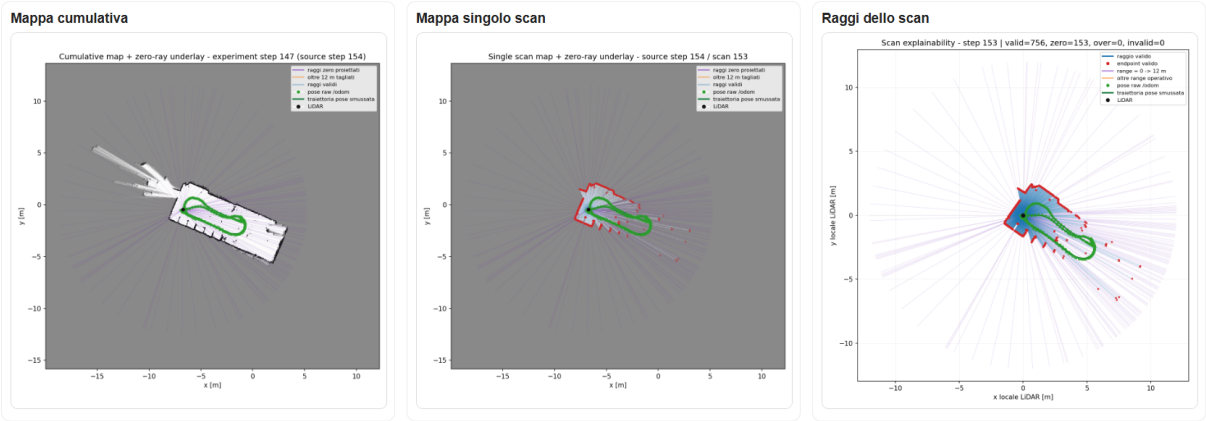


Figura 8: Viewer step-by-step in una fase avanzata della sequenza. La mappa cumulativa mostra l'effetto dell'accumulo progressivo delle osservazioni.

Nella Fig. 8 è mostrata una fase avanzata della stessa sequenza. In questo caso la differenza tra mappa cumulativa e singola osservazione è molto più evidente. La singola scansione continua a descrivere solo ciò che il LiDAR osserva in quell'istante, mentre la mappa cumulativa conserva l'informazione raccolta dagli scan precedenti. Di conseguenza, le regioni libere e i bordi occupati risultano più estesi e più stabili.

Anche in questo caso la vista dei raggi consente di controllare la qualità dello scan corrente. Nell'esempio avanzato sono presenti 756 raggi validi e 153 raggi a zero, senza raggi oltre il range operativo. Questo conferma che una parte consistente delle misure grezze può essere non informativa, ma viene correttamente esclusa dall'aggiornamento della griglia.

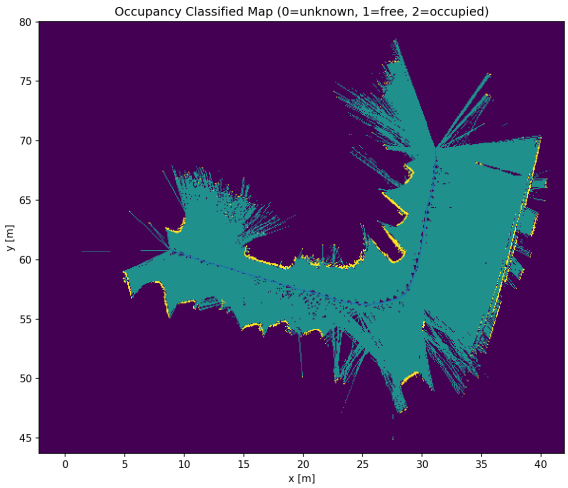
Dal punto di vista dell'algoritmo, ogni raggio valido produce due tipi di aggiornamento: le celle attraversate lungo il percorso vengono aggiornate come *free*, mentre la cella finale viene aggiornata come *occupied*. Per questo motivo il numero di aggiornamenti *free* è generalmente molto maggiore del numero di aggiornamenti *occupied*: un singolo raggio può attraversare molte celle libere, ma termina in una sola cella occupata.

La viewer permette quindi di verificare concretamente il funzionamento della pipeline: mostra quali raggi contribuiscono alla mappa, quali vengono esclusi, come si distribuiscono gli endpoint occupati e come le osservazioni successive rafforzano progressivamente la struttura finale della occupancy grid.

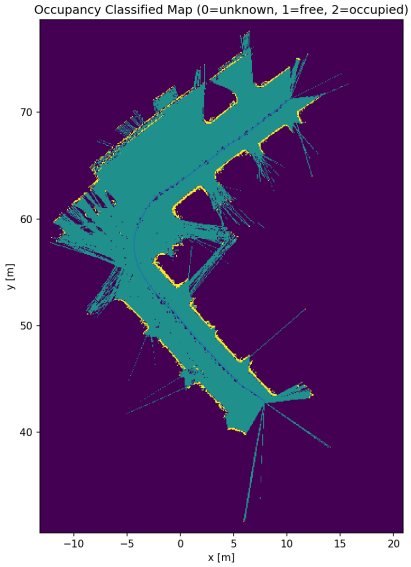
6.5 Esempi aggiuntivi in ambiente esterno

Oltre al caso studio indoor analizzato nelle sezioni precedenti, la pipeline è stata applicata anche ad alcune registrazioni acquisite in ambiente esterno. Le mappe riportate in questa sezione non vengono analizzate nel dettaglio, ma sono inserite a scopo esemplificativo per mostrare come lo stesso metodo possa essere applicato a scenari diversi.

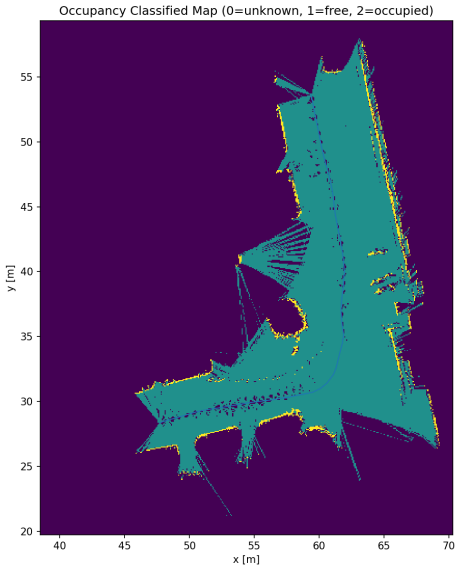
In ambiente esterno la struttura della mappa può risultare meno regolare rispetto al caso indoor, poiché sono presenti spazi più ampi, ostacoli distribuiti in modo meno ordinato e zone in cui il LiDAR può ricevere misure più sparse. Tuttavia, la logica di costruzione rimane la stessa: le celle attraversate dai raggi vengono aggiornate come libere, mentre gli endpoint validi contribuiscono alle celle occupate.



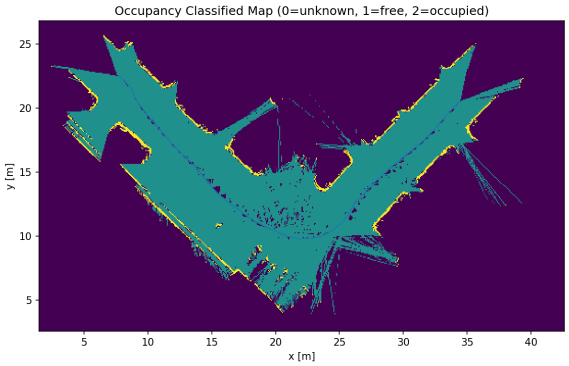
(a) Esempio 1



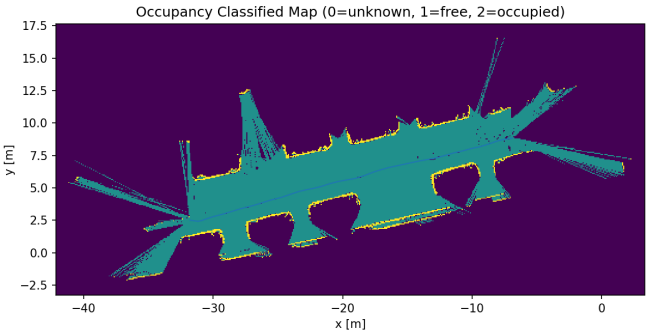
(b) Esempio 2



(c) Esempio 3



(d) Esempio 4



(e) Esempio 5

Figura 9: Esempi aggiuntivi di occupancy grid ottenute da registrazioni in ambiente esterno.

7 Conclusioni

Il lavoro ha permesso di costruire una pipeline offline per la generazione di occupancy grid 2D a partire da dati reali acquisiti con LiDAR e odometria ZED. La pipeline separa in modo chiaro preprocessing, mapping e strumenti di debug, rendendo possibile analizzare sia il risultato finale sia il contributo delle singole osservazioni.

Il caso studio indoor mostra che la mappa ottenuta è qualitativamente coerente con la struttura dell'ambiente osservato: le celle libere si sviluppano lungo le regioni attraversate dai raggi, mentre gli endpoint ricorrenti formano bordi occupati. Allo stesso tempo, i dati reali evidenziano alcune limitazioni, legate soprattutto alla presenza di misure LiDAR nulle, alla calibrazione approssimata tra sensori e alla stabilità della posa.

Gli strumenti di viewer ed explainability hanno permesso di interpretare meglio il processo di costruzione della mappa, distinguendo il contributo dei raggi validi da quello delle misure non utilizzate. Possibili sviluppi futuri includono una calibrazione più accurata tra ZED e LiDAR, l'introduzione del deskew raggio per raggio e una valutazione più sistematica dei parametri di mapping.